ENGINEERING HANDOFF DOCUMENT

RAMP HRMS

Recruitment And Management of People

Full-stack MERN HR Management System — Architecture & Project-Status Overview

Purpose	A complete handoff so any developer or AI can read this, understand the system, and continue the work.
Scope	Backend (Express + MongoDB, 19 feature modules) and Frontend (React 19 + Vite). Derived from a full read of the source.
Stack	MERN — MongoDB, Express, React 19, Node.js (ESM). Redux Toolkit, TanStack Query, Tailwind, Zod, JWT.
Repo	Monorepo: /backend and /frontend. Owner: ITSYBIZZ AI Private Limited.
Date	2026-07-21
Status	Feature-complete demo; core lifecycle works end to end. Several security gaps and a few stub/partial features remain (Section 11-12).

Note: this document records both what is built and what is incomplete or risky, so the next contributor starts with an accurate picture rather than an optimistic one.

Contents

1 Executive Summary

RAMP (Recruitment And Management of People) is a production-styled Human Resource Management System that covers the full employee life-cycle: recruitment and onboarding, attendance, leave, payroll, performance, assets, reimbursements, documents, announcements, holidays, and offboarding, all under a four-tier role-based access model with an immutable audit trail.

The backend is a modular Express + MongoDB (Mongoose) REST API written in modern ESM, organised one folder per feature with a clean **routes -> controller -> service -> repository -> model** layering. The frontend is a React 19 + Vite single-page app using Redux Toolkit for auth/UI state, TanStack Query for all server state, React Hook Form + Zod for forms, and a Tailwind design system ported 1:1 from a single-file reference prototype (`itsybizz-hrms (2).html`).

Maturity. The happy-path of every module works: you can seed a database, log in as any of four roles, and exercise the whole lifecycle. What remains is hardening: a set of backend authorization gaps (the UI hides features by role, but several read endpoints only check authentication), a few features that are stubs or partially wired (notably the asset external-API sync and offer salary-structure application), and normal correctness/quality polish. These are catalogued precisely in Sections 11 and 12.

READ ME

How to use this document

Sections 2-10 explain the architecture and how the code is organised. Section 11 is the done / partial / missing status table. Section 12 lists concrete bugs and security issues with file references and severity. Section 14 is a prioritised next-steps backlog. Every file path is relative to the repository root.

2 Technology Stack

Layer	Technologies
Backend runtime	Node.js 18+ (ES Modules, "type": "module"), Express 4
Database	MongoDB with Mongoose 8 (ODM). Single Atlas/local connection.
Validation	Zod schemas (body / params / query) via a reusable validate() middleware
Auth & crypto	jsonwebtoken (access + refresh + reset), bcryptjs password hashing
Security	helmet, cors (credentials), express-rate-limit, express-mongo-sanitize, xss
Files & mail	multer (uploads), nodemailer (password-reset email)
Logging	winston (app log) + morgan (HTTP log) piped into winston
Frontend core	React 19.2, Vite 8, React Router 6 (lazy routes)
Client state	Redux Toolkit 2 (auth + UI only); TanStack Query 5 (all server state)
Forms & UI	React Hook Form 7 + Zod, Tailwind 3, Framer Motion, Chart.js 4, React Icons
HTTP	axios instance with bearer-token injection + transparent 401-refresh interceptor

3 Repository Layout

The repo is a two-package monorepo plus three top-level docs and the original design prototype.

```
HR-Management-system/
|- API_DOC.md            Full REST reference (endpoints, payloads, status codes)
|- GUIDE.md              Feature guide (setup + how to use each module by role)
|- README.md            Project intro + quick start
|- itsybizz-hrms(2).html Original single-file RAMP design prototype (UI source of truth)
|- backend/
|   +- src/
|      |- app.js         Express app: security, parsing, routes, error handling
|      |- server.js      Bootstrap: config check -> DB connect -> listen -> graceful shutdown
|      |- config/        Validated env config + Mongoose connection
|      |- common/        constants, middlewares, shared models (Counter), utils, RBAC
|      |- routes/index.js Central mount table (/api/v1)
|      |- seed/          Reference dataset + `npm run seed`
|      +- modules// Per feature: model . repository . service . controller . validation . routes
+- frontend/
    +- src/
       |- main.jsx       Providers: Redux, React Query, Router
       |- App.jsx        Route table (lazy pages + ProtectedRoute + RoleRoute)
       |- index.css      Design tokens + ported component classes
       |- api/index.js   Typed API client (one object per resource)
       |- lib/           axios instance/interceptors, queryClient, charts
       |- store/         Redux store, authSlice, uiSlice
       |- constants/nav.js  Navigation, role->views, permissions
       |- hooks/ utils/ components/ui/ layouts/ routes/
       +- features//  Page.jsx + feature components/modals
```

4 Backend Architecture

4.1 Layered structure

Every feature module is a self-contained folder with the same five layers. Controllers only validate, delegate and respond; services hold business logic and write the audit log; repositories/models do persistence. There is no per-controller try/catch — an `asyncHandler` wraps every handler and forwards errors to one central error middleware.

routes.js -> **controller** -> **service** -> **repository** -> **model**

validate + delegate + respond | business logic + audit | Mongoose persistence

4.2 Request lifecycle

- **Global middleware** (app.js): trust proxy -> helmet (CORP cross-origin) -> cors with credentials -> JSON/urlencoded (2mb) -> cookie-parser -> compression -> mongo-injection guard -> XSS guard -> morgan logging -> static /uploads -> /health -> /api/v1 behind the rate limiter.
- **Authentication** (auth.middleware): reads a Bearer token (or accessToken cookie), verifies it, then re-loads the live User from the DB so a deactivated or deleted account is rejected even with an otherwise-valid token. Populates req.user = {id, role, empId, name, email}.
- **Authorization**: requirePermission('x') checks the PERMISSIONS map; authorizeRoles(...) and requireView('view') are also available. Enforced per-route in each module's routes.js.
- **Validation**: validate({body,params,query}) parses with Zod and writes the coerced values back onto the request so handlers get clean, typed data.
- **Response**: handlers return ApiResponse.ok(res, data, message) -> uniform { success, message, data, meta? } envelope. Errors throw ApiError; the error middleware normalises Mongoose ValidationError / CastError / duplicate-key / JWT errors and hides stack traces in production.

4.3 Cross-cutting infrastructure (common/)

Piece	What it does
ApiResponse / ApiError	Uniform success envelope; error factories (badRequest, unauthorized, forbidden, notFound, conflict, unprocessable).
asyncHandler	Wraps async handlers so thrown/rejected errors reach the central error middleware. No try/catch in controllers.
Counter model + nextId()	Atomic sequence store for human-readable IDs: EMP001, LV-1044, JOB-07, OFR-01, EX-001, etc. Each module seeds a start offset.
rbac.js	ROLE_VIEWS (role -> allowed views) and can(role, perm). Mirrors the frontend nav.js so both layers agree.
query.js	parsePagination (page/limit/sort, max 100), buildMeta (pagination envelope), buildSearch (regex-escaped case-insensitive OR search).
salary.js	breakup(gross) statutory split, inWords(n) Indian numbering, money(n) Rs. formatting. Used by payroll + offers.
enrich.js	Batch employee lookups ({empId,name,dept,role,status}) to hydrate string-keyed records and avoid N+1 queries.
jwt.js / password.js	Sign/verify access, refresh, reset tokens; bcrypt hash/compare.
mailer.js / logger.js	Nodemailer transport (password reset); winston logger + morgan stream.
Middlewares	auth, validate, error, rateLimit (stricter authLimiter on /auth), sanitize (mongo + xss), upload (multer).

4.4 Security posture (what is good)

- Mongo-operator sanitisation and XSS cleaning applied globally before any route runs.
- Search is regex-escaped (buildSearch) and sort keys are used as plain field names, so no $regex or operator injection.
- Passwords bcrypt-hashed; password / refreshTokens / isActive are select:false by default; tokens re-validated against the live user every request.
- Refresh-token rotation with a small retained history; reset/change-password wipe all refresh tokens to force re-login everywhere.
- Settings general-update uses an allow-list, so the broad PUT cannot mass-assign the asset-API block or letter templates (those have dedicated, HR-Admin-only endpoints).
- Consistent audit trail (fire-and-forget, self-swallowing) on all significant mutations; soft deletes via deletedAt.

Gaps in this posture (unauthenticated /uploads, missing authorization on several read endpoints, plaintext asset-API key readable by all) are detailed in Section 12.

5 Roles & Permissions (RBAC)

Four roles, enforced at two layers: the API (permission middleware) and the UI (route guards + filtered navigation). The same permission/ view maps exist on both sides so they stay in agreement. Y = full use, V = view/own only, dash = hidden.

Module / View	HR Admin	HR Rep	Finance	Employee
Dashboard	Y	Y	Y	Y
Employees	Y	Y	V	-
Attendance	Y	Y	-	V
Leave	Y	Y	-	V
Payroll	Y	-	Y	V
Exit / Offboarding	Y	Y	V	V
Recruitment (+offers = Admin)	Y	Y	-	-
Performance	Y	Y	-	-
Assets	Y	Y	-	-
Reimbursements	Y	-	Y	V
Documents	Y	Y	V	V
Announcements	Y	Y	-	V
Holidays	Y	Y	-	V
Departments	Y	Y	-	-
Reports	Y	Y	Y	-
Settings	Y	-	-	-

Permissions (fine-grained, permission -> roles): manageEmployee, approveLeave, recruit, manageGoals, assignAsset, verifyDoc, announce, manageDepartment, manageHoliday, manageExit = HR Admin + HR Rep; approveExpense, clearExpense, runPayroll = HR Admin + Finance; clearExit = HR Admin + HR Rep + Finance; offer, salaryStructure, settings = HR Admin only.

6 Data Models & Relationships

Roughly 20 Mongoose collections. Most models carry a human-readable **code** (from the Counter), a **deletedAt** soft-delete field, and timestamps. A crucial design point for the next contributor: **relationships are string-keyed** (by empId, by code, or even by free-text title) rather than ObjectId refs — there are no populated joins and no referential-integrity/cascade guarantees. Renaming or deleting a parent does not propagate.

Model	Key fields	Links by
User	empId, name, email, password(hashed, hidden), role, isActive, mustResetPassword, refreshTokens(hidden), lastLogin	empId + employee ObjectId
Employee	empId, name, dept, role(designation), email, salary, gender, status(Active/Inactive/Exited), access(RBAC role), managerId	1:1 User (via empId)
Department	code, name(unique), head, headEmpId, description	employees by dept name
Attendance	emp, date(YYYY-MM-DD), st(P/A/L/W/H/''), in, out. Unique index (emp,date)	emp = empId
Leave	code(LV-1044), emp, type(Casual/Sick/Earned), from, to, days, status, approver, decidedAt	emp = empId

Model	Key fields	Links by
PayrollRun	month(YYYY-MM, unique), status(Pending/Processing/Paid), paidOn, paidEmps[]	month + empIds
Opening / Candidate	Opening: code, title, dept, positions, status. Candidate: code, name, job(title text), stage	Candidate.job = Opening title (text)
SalaryStructure / Offer	Structure: basicPct/hraPct/specialPct, pf, pt, gratuity. Offer: candidate, ctc, joinDate, structureCode, letter	Offer.structure Code (stored, unused)
Goal / Review	Goal: emp, title, due, progress(0-100). Review: emp, cycle, rating(1-5), note	emp = empId
Asset	code, name, type, tag, emp(''=free), status(Assigned/Available/In Repair), since, src(api/manual)	emp = empId
Expense	code, emp, title, cat, amt, date, status(Pending/Approved/Rejected/Paid), decidedBy	emp = empId
Document	code, emp, name, type, status(Pending/Verified/Rejected), filePath(/uploads/documents/...), size	emp = empId
Announcement / Holiday	Announcement: code, title, body, by, pin. Holiday: date(unique), name	-
Notification	t, s, ico, link, user(empId; ''=broadcast), read	user = empId or '' broadcast
Exit / ExitDoc	Exit: code, emp, type, reason, lastDay, status, clearance{IT,Finance,Admin,HR,Reporting}, interviewDone, fnfAmount, fnfStatus. ExitDoc: exit(code), dir(upload/generated), letter, filePath	ExitDoc.exit = Exit code (text)
Settings (singleton)	company profile, leave quotas cl/sl/el, weekOff[], inTime/lateAfter, toggles, assetApi{url,key,enabled,lastSync}, offerTemplate, exitTemplates	key='app'
AuditLog	action, entity, entityId, actorId/name/role, description, meta, ip, createdAt	actorId ref User

7 Module Catalog & Completion Status

All 19 backend modules have the full five-layer implementation and a matching frontend page. The status column reflects functional completeness of the happy path; caveats point to Section 12.

Module	Status	Notes / caveats
Auth	Done	JWT access+refresh rotation, forgot/reset/change password, /me profile with role views. Solid.
Users	Done	List/search, change role (syncs Employee.access), activate/deactivate. No last-admin guard.
Employees	Done	CRUD in a transaction that auto-provisions the linked login; toggle status, soft delete, directory, CSV export.
Departments	Done	CRUD with live headcount aggregation; delete blocked if staffed.
Attendance	Done	Check-in/out, HR mark, monthly grid with holiday/week-off precedence. Server-local time; some counts quirks.
Leave	Done	Apply/approve/reject/withdraw, CL/SL/EL balances from approved leaves. No overlap/balance/date-order checks; self-approval possible.
Payroll	Partial	Monthly run, per-employee pay, payslip with statutory breakup + amount-in-words. Amounts NOT snapshotted (recomputed live); read endpoints under-guarded.
Recruitment	Partial	Openings, candidate pipeline, salary structures, offer-letter generation. structureCode stored but never applied; offers decoupled from pipeline; offers not deletable.
Performance	Done	Goals with progress bump/set, reviews with 1-5 rating. Row-scoped for Employee role.
Assets	Partial	Assign/return/repair, summary counts, manual add. 'Sync from external API' is a STUB (no HTTP call is made).
Expenses	Done	Claim -> approve/reject -> pay with an enforced state machine; per-status summary. Cleanest module.
Documents	Partial	Upload (multer), verify/reject, vault. emp can be spoofed on upload; no verify/reject state guard; files served publicly.
Announcements	Done	Create/edit/pin/delete; board + dashboard feed.
Holidays	Done	Add/remove, year filter, upcoming feed; duplicate-date guard.
Notification	Partial	Own + broadcast feed, mark read / read-all / clear. Broadcast read-state is a single shared flag (one user marks it read for everyone).
Exit	Partial	Initiate, 5-point clearance, interview, F&F settle, letter generation, complete/withdraw. complete has no preconditions; no terminal-state guards; read endpoints ungated (IDOR).
Reports	Partial	Overview KPIs, headcount, salary bands, attendance trend (aggregations). Endpoints have NO authorization guard.
Settings	Done	Company profile, leave policy, offer/exit templates, asset-API config. GET returns assetApi.key to all authenticated users.
Audit	Done	Immutable log of mutations; HR-Admin-only list with filters. ip almost always null (callers omit it).

8 API Surface

Base URL `/api/v1`. All routes require a Bearer access token except /auth/login, /auth/refresh, /auth/forgot-password and /auth/reset-password. Full details live in API_DOC.md; this is the map.

Module	Endpoints
auth	POST login | refresh | logout | change-password | forgot-password | reset-password ; GET me
users	GET / (manageEmployee) | GET /me ; PATCH :id/role (settings) | :id/activate | :id/deactivate
employees	GET / | GET :id | POST | PUT :id | DELETE :id | PATCH :id/toggle-status | GET directory
departments	GET / | GET :id | POST | PUT :id | DELETE :id
attendance	GET today | summary | month ; POST check-in | check-out | mark
leaves	GET / | GET balance ; POST / ; PATCH :id/approve | :id/reject ; DELETE :id
payroll	GET / | months | payslip/:empId ; POST run | pay
recruitment	GET/POST/PUT/PATCH/DELETE openings | candidates(+/:id/stage) | salary-structures ; GET/POST offers | GET offers/:id
performance	GET goals | reviews ; POST goals | reviews ; PATCH goals/:id/progress ; DELETE goals/:id | reviews/:id
assets	GET / | summary ; POST / | sync ; PUT :id ; PATCH :id/assign | return | repair-done ; DELETE :id
expenses	GET / | summary ; POST / ; PATCH :id/approve | reject | pay ; DELETE :id
documents	GET / | GET :id ; POST / (multipart) ; PATCH :id/verify | reject ; DELETE :id
announcements	GET / ; POST / ; PUT :id ; PATCH :id/pin ; DELETE :id
holidays	GET / | upcoming ; POST / ; DELETE :id
notifications	GET / ; POST / (announce) ; PATCH :id/read | read-all ; DELETE / (clear)
exit	GET / | :id | :id/documents | documents/:docId ; POST / | :id/documents/generate ; PATCH :id/clearance | interview | fnf | settle-fnf | withdraw | complete ; DELETE :id
reports	GET overview | headcount | salary-bands | attendance-trend
settings	GET / ; PUT / | offer-template | exit-templates | asset-api
audit	GET / (settings)

9 Key Business Logic

9.1 Authentication & tokens

- login verifies bcrypt password, issues an access token (15m) carrying {id, role, empId} and a refresh token (7d); the refresh token is stored on the user (last ~5 kept) and also set as an httpOnly cookie.
- refresh rotates: the used refresh token is dropped and a new pair issued. Reset/change-password clear all refresh tokens.
- buildProfile returns the user plus allowedViews(role), so the client renders role-appropriate navigation immediately.
- forgotPassword always returns success (anti-enumeration); a reset link is emailed only if the account exists.

9.2 Payroll & salary breakup (common/utils/salary.js)

breakup(gross) computes a simplified statutory split. These are model formulas, not audit-grade statutory calculations:

```
basic   = round(gross * 0.50)
hra     = round(gross * 0.20)
special = gross - basic - hra          (absorbs rounding, ~30%)
pf      = basic >= 15000 ? 1800 : round(basic * 0.12)
pt      = gross > 21000 ? 200 : 0
tds     = gross > 90000 ? round(gross*0.06)
        : gross > 60000 ? round(gross*0.03) : 0
net     = gross - (pf + pt + tds)
```

Critical caveat: PayrollRun stores only run-level state (month, status, paidEmps). No per-employee amounts are persisted, so payslips are recomputed live from the current employee salary — changing a salary retroactively rewrites historical payslips. Snapshotting is a recommended fix (Section 14).

9.3 Leave & balances

- dayCount(from,to) is an inclusive calendar-day count (does not exclude weekends/holidays); a reversed range silently collapses to 1 day.
- balance uses per-year quotas from Settings (defaults CL 12, SL 10, EL 18) minus the sum of APPROVED-only leave days of each type in the current year. Pending leaves are not counted; the result can go negative (no clamp).
- Apply always uses the caller's own empId. There is no balance check, no overlap detection, and no separation-of-duties guard (an approver could approve their own leave).
- Approving a leave does NOT write an 'L' attendance record, so the attendance grid and leave records can disagree.

9.4 Attendance

- One row per (emp, date) enforced by a unique index; check-in/out upsert the row. Times use server-local time (payroll/exit use UTC -- a cross-module inconsistency).
- Monthly grid precedence per day: stored status wins, else Holiday, else Week-Off (Settings.weekOff, default Sun), else Not-Marked. The counts object omits a Not-Marked bucket, so counts do not reconcile to days-in-month.

9.5 Exit / offboarding state machine

- States: In Progress -> Clearance -> Completed / Withdrawn. All 5 clearances true auto-advances In Progress to Clearance.
- complete sets Employee.status = Exited and disables the login (User.isActive = false). This is the consequential transition.
- Weaknesses: complete enforces NO preconditions (works with 0/5 clearances, no interview, unpaid F&F); there are no terminal-state guards, so withdraw on a completed exit leaves the employee Exited/login-disabled while the case reads 'Withdrawn'.

10 Frontend Architecture

10.1 Bootstrap, routing, auth flow

- Provider tree: StrictMode -> Redux Provider -> QueryClientProvider -> BrowserRouter -> App. App adds ConfirmProvider (imperative confirm dialog) and renders ToastHost.
- Routing: /login is eager; the 16 feature pages are React.lazy + Suspense. Everything is wrapped ProtectedRoute (auth) -> DashboardLayout -> RoleRoute view={view} (redirects to /dashboard if the role cannot view it).
- Auth: access token lives in Redux and localStorage (key 'ramp_auth') and is injected as a Bearer header; the refresh token rides the httpOnly cookie. A singleton 401 interceptor dedupes concurrent refreshes, replays the original request, and logs out on failure. There is no boot-time /auth/me revalidation.

10.2 State management split

- Redux (only two slices): authSlice (user, accessToken, isAuthenticated -- persisted) and uiSlice (sidebarOpen, toasts, theme).
- TanStack Query owns ALL server state: retry 1, refetchOnWindowFocus off, staleTime 30s. Pages read data.data / data.meta and invalidate on mutation. No server entities are kept in Redux.

10.3 Typed API client (api/index.js)

A thin wrapper over the shared axios instance, organised as one exported object per resource (authApi, employeeApi, leaveApi, payrollApi, recruitmentApi, ... ~20 in total). unwrap(p) returns the response body; qs(params) builds a query string dropping empty values. REST conventions: GET list/detail, POST create, PUT update, PATCH transitions, DELETE remove; documentApi.create posts multipart.

10.4 UI kit & the standard page pattern

components/ui/ is a small design system: DataTable, Modal (portaled), Field + inputs, Button, Card, StatCard, Chart (imperative Chart.js wrapper), ConfirmProvider/useConfirm, ToastHost, Tabs, Pagination, Chip, Avatar, EmployeeCell, Icon (inline SVG set), PageHeader, Spinner, ThemeToggle. Almost every list page follows one template:

- Hooks header: useAuth (can(perm), empId, isEmployee), useToast, useQueryClient, useConfirm.
- Debounced search + a filters object; paginated useQuery(placeholderData: keepPreviousData) plus summary/directory queries.
- Mutations toast + invalidateQueries on success, toast(apiError(e),'error') on failure; destructive actions gated behind an await confirm({danger:true}).
- Render: PageHeader (role-gated buttons) -> StatCards -> Card/filter-bar -> DataTable (role-gated action column) -> Pagination -> modals.

Charts: only the Dashboard uses Chart.js (attendance line/bar toggle, gender doughnut, headcount bar, theme-aware). Reports re-implements the same visuals with hand-rolled CSS/SVG -- a duplication worth unifying.

NOTE

Client-side RBAC is UX only

RoleRoute / canView / can gate what the UI shows, but they are trivially bypassable. Real enforcement must live on the backend. Section 12 lists the read endpoints where that backend enforcement is currently missing.

11 What Is Done vs Partial vs Missing

11.1 Working end to end (Done)

- Seed + login as four roles; role-filtered navigation and route guards.
- Employee CRUD with auto-provisioned login accounts (transactional); departments with live headcount.
- Attendance check-in/out and HR marking; leave apply/approve/reject/withdraw with computed balances.
- Payroll run + payslip generation with statutory breakup and amount-in-words; CSV export.
- Recruitment openings/candidates/structures and offer-letter generation; performance goals + reviews.
- Assets assign/return/repair; reimbursement claim/approve/pay; document upload/verify; announcements; holidays; audit log; company settings; dashboard + reports analytics; exit workflow with letter generation.

11.2 Partial / stubbed (needs work)

- **Asset external-API sync is a stub** -- despite the docs, sync() makes no HTTP request (there is no HTTP client in the backend); it only stamps lastSync and counts pre-existing api-sourced rows.
- **Offer salary structure not applied** -- Offer.structureCode is stored but never used; the letter shows only total CTC. Offers are also not linked to the candidate pipeline and cannot be deleted.
- **Payroll amounts not snapshotted** -- historical payslips mutate with current salary.
- **Exit document upload path** -- the ExitDoc model supports dir:'upload' + filePath, but there is no upload route (only generation).
- **Broadcast notifications** -- share a single read flag across all users.
- **Password-reset email** -- only works if SMTP_* is configured; the LoginPage 'Forgot password?' link just shows a contact-HR toast, and 'Remember me' is non-functional.

11.3 Not present (by design, for now)

- No automated tests (no unit/integration/e2e suites), no CI config.
- No file-storage abstraction (uploads go to local disk under src/uploads).
- No pagination/observability dashboards, feature flags, or i18n.
- No real external integrations (asset API, payroll bank files, HRIS import).

12 Known Issues & Risks

Precise, file-referenced findings from a full read. Fix the security items before any real deployment. Severity: HIGH = exploitable data exposure; MED = correctness/logic with user impact; LOW = quality/DRY.

12.1 Security

HIGH

Uploads served publicly, unauthenticated (backend/src/app.js)
app.use('/uploads', express.static(...)) with helmet CORP cross-origin exposes every employee document at /uploads/documents/ to anyone who knows or guesses the URL, bypassing the owner/verifier checks in the document service. The upload filter is extension-only (no MIME/content check), so a renamed HTML/SVG passes and could be served same-origin. Highest-impact issue.

HIGH

Missing authorization on read endpoints
Several GET routes are gated by authenticate only: reports (overview/headcount/salary-bands/attendance-trend) expose company-wide salary/payroll analytics to any employee; GET /payroll, /months, /payslip/:empId let non-employee roles read everyone's full salary breakup; all exit read endpoints (list, :id, documents) are ungated (IDOR on sequential EX-/DX- codes, exposing F&F, reasons, termination/relieving letters). Candidates, openings, salary-structures, assets and performance GETs are similarly broad.

HIGH

Asset-API token world-readable; secrets in example env
GET /settings returns the whole singleton including assetApi.key (plaintext) to any authenticated user. Separately, backend/.env.example (tracked in git) ships a real-looking MongoDB Atlas connection string with embedded credentials -- rotate those and replace with a placeholder. (.env itself is correctly gitignored.)

MED

Document emp-spoofing on upload; latent SSRF
The document create route has no permission guard and trusts an optional emp field, so any employee can attribute a document to another person's empId. Also, assetApi.url is stored with zero validation -- harmless today because sync() never fetches it, but a trap once a real fetch is wired in (could point at internal services / cloud metadata).

12.2 Correctness

MED

Payroll historical payslips are mutable
No snapshotting; payslips recompute from current salary. Past payroll is not an immutable record.

MED

Exit state machine has no guards
complete() checks nothing (clearances/interview/F&F all advisory); no terminal-state guard, so withdraw on a completed exit leaves the employee Exited/login-disabled while the case reads Withdrawn. remove() does not cascade to ExitDocs or reactivate the employee.

MED

Leave gaps + broadcast-notification shared read flag
Leave apply lacks balance/overlap/date-order validation and allows self-approval; approved leave does not write attendance. Broadcast notifications share one read flag, so one user marking a broadcast read flips it for everyone, and 'Clear all' never removes broadcasts.

MED

Timezone + per-page-total inconsistencies

Attendance uses server-local time while payroll/exit use UTC (month/day boundary bugs). Several UI StatCards ('Active', 'Pending') are computed from the current page's rows, not meta.total, so they mislead once data spans multiple pages.

12.3 Quality / maintainability

- Access token in localStorage (XSS-exfiltratable); no boot-time session revalidation (brief authenticated flash on an expired token).
- Dead code: React Query Devtools installed but never mounted; refreshMe / authApi.me unused; PAYROLL_STATUS.PROCESSING unused; unreachable RoleRoute params.view branch.
- Duplication: Skeleton copied across 4 pages; filter-bar/search markup repeated on ~8 pages; gender donut + headcount bar exist in two implementations (Chart.js vs SVG); blob-download helper re-implemented 3x.
- String-keyed relationships throughout: no refs/cascades -- renaming a department or deleting a candidate leaves dependents dangling. Counter has a tiny lost-update race only on the very first id per key.
- audit.record is never awaited (fire-and-forget) and its ip argument is almost always omitted by callers.

13 Setup & Run

```
# Backend
cd backend
cp .env.example .env      # set MONGODB_URI + JWT secrets
npm install
npm run seed              # wipe + load reference data (18 employees, etc.)
npm run dev               # API on http://localhost:5000/api/v1

# Frontend
cd frontend
npm install
npm run dev               # app on http://localhost:5173
#   NOTE: vite.config.js currently proxies /api and /uploads to the LIVE
#   Render backend, so `npm run dev` reads/writes PRODUCTION data unless changed.
```

Health check: GET http://localhost:5000/health returns { success:true, status:'ok' }. Required env: MONGODB_URI, JWT_ACCESS_SECRET, JWT_REFRESH_SECRET (the app fails fast if missing).

14 Recommended Next Steps (prioritised)

A suggested backlog for whoever picks this up, ordered by value/effort.

#	Task	Why
1	Gate /uploads behind auth + ownership (or signed URLs); add MIME/content validation to the upload filter.	Closes the highest-impact data-exposure hole.
2	Add requireView/ requirePermission to reports, payroll, exit and other broad GET endpoints; row-scope by empId.	Stops cross-employee salary/exit data leakage.
3	Mask assetApi.key in GET /settings; rotate the credentials committed in .env.example.	Removes secret exposure.
4	Snapshot payroll amounts into PayrollRun at run time.	Makes payslips immutable, audit-safe records.
5	Harden the exit state machine (preconditions on complete, terminal guards, cascade on remove).	Prevents inconsistent Exited/Withdrawn states.
6	Fix broadcast notifications (per-user read state) and leave validation (balance/overlap/date-order/self-approval).	Removes user-visible correctness bugs.
7	Implement the real asset-API sync (add an HTTP client, validate the URL against SSRF) OR relabel it honestly.	Matches behaviour to documentation.
8	Apply Offer.structureCode to the generated letter; link offers to the candidate pipeline.	Completes the recruitment feature.
9	Add a test suite (services first) and a CI pipeline; extract shared UI (Skeleton, FilterBar, download helper).	Long-term maintainability.

Appendix A Seed Accounts

After npm run seed, every seeded login uses the SEED_DEFAULT_PASSWORD (default Password@123).

Role	Email
HR Admin (full access)	`pankaj@itsybizz.com`
HR Representative	`pooja@itsybizz.com`
Finance Representative	`priya@itsybizz.com`
Employee	`rohit@itsybizz.com`

Appendix B Environment Variables

Variable	Purpose
`MONGODB_URI`	Mongo connection string (required)
`JWT_ACCESS_SECRET / _EXPIRES_IN`	Access-token signing (required) / lifetime (default 15m)
`JWT_REFRESH_SECRET / _EXPIRES_IN`	Refresh-token signing (required) / lifetime (default 7d)
`JWT_RESET_SECRET / _EXPIRES_IN`	Password-reset token / lifetime (default 15m)
`CLIENT_URL`	Allowed CORS origin(s), comma-separated
`BCRYPT_SALT_ROUNDS`	Password hash cost (default 10)
`RATE_LIMIT_* / AUTH_RATE_LIMIT_MAX`	API + auth rate-limit windows/maxes
`SMTP_* / MAIL_FROM`	Nodemailer transport for password-reset email (optional)
`UPLOAD_DIR / MAX_FILE_SIZE_MB`	Upload directory + size cap (default 10 MB)
`SEED_DEFAULT_PASSWORD`	Password every seeded login receives (default Password@123)

End of handoff document — RAMP HRMS. Generated from a complete source read on 2026-07-21. For endpoint-level payloads see API_DOC.md; for step-by-step feature usage see GUIDE.md.